Relatório de Evolução do Projeto

ChargeGrid Assistant — GoodWe EV Challenge 2026 — Sprint 04 (RAG)

1. Resumo da evolução do projeto

O ChargeGrid Assistant é um chatbot de suporte operacional para eletropostos comerciais, desenvolvido em sprints cumulativas em branches próprias do mesmo repositório:

- **Sprint 1 — planejamento:** definição do problema (gestão de múltiplos carregadores sem mecanismos integrados), persona (operador comercial de eletroposto), proposta e seleção de tecnologias.
- **Sprint 2 — chatbot funcional:** loop de terminal em Python com Ollama, memória por histórico de mensagens, system prompt com contexto simulado do eletroposto e few-shot prompting.
- **Sprint 3 — refactory para LangChain:** núcleo reescrito em LCEL (prompt | llm | parser), memória por sessão com limite de tokens, saída estruturada em Pydantic v2 e guardrails de segurança (validação de escopo + detecção de jailbreak/prompt injection).
- **Sprint 4 — RAG + interface:** busca semântica (RAG) sobre base de conhecimento oficial em PDFs, com ChromaDB persistente, embeddings locais, citação de fonte (documento + página), interface Streamlit e avaliação formal com 19 casos + LLM-as-judge.

2. Arquitetura construída na Sprint 04

Etapa	Implementação	Decisão principal
Base de conhecimento	4 PDFs oficiais (manual, regimento, FAQ e tabela tarifária)	Gerados artificialmente, curtos e densos, mantendo consistência com os dados simulado
Loader	PyMuPDFLoader, 1 documento por página	Preserva a origem de cada trecho (source + page), base da citação
Chunking	RecursiveCharacterTextSplitter 500/50	Chunks pequenos evitam misturar tópicos em documentos densos
Embeddings	nomic-embed-text via Ollama local	A API do Ollama Cloud não expõe embeddings (issue ollama/ollama#17129)
Vector store	ChromaDB persistente (data/chroma_db/)	Coleção recriada a cada indexação — corrige duplicações
Retriever	similarity_search com k=4	4 chunks equilibram contexto e custo de tokens
Prompt	prompt_rag_v1/v2 versionados	Exige citar 'Fonte: doc (página X)' e frase fixa de recusa
Pipeline	src/rag/pipeline.py	Integra guardrails da Sprint 03 (injection/fora de escopo) ao RAG
Interface	Streamlit (app/app.py)	Citação de fonte por resposta, histórico e cache de recursos

3. Metodologia de avaliação

Eval set de **19 casos** (11 factual, 2 multi-documento, 2 fora de contexto, 2 sem resposta na base, 2 injection no documento) com métricas determinísticas (chunk da fonte certa recuperado, citação presente, fonte correta citada, fatos-chave presentes, injection resistido) e **LLM-as-judge** (faithfulness e answer relevancy, com prompt e rubrica próprios — fallback manual do RAGAS, pois a biblioteca não instala junto com a versão atual do langchain-community). Modelo gerador e avaliador: gpt-oss:120b (temperature 0); juiz com max_tokens=1500. Resultados e comparações geradas por evals/run_rag_evals.py e evals/analisar_interacao.py.

4. Tabela antes/depois (evolução medida na sprint)

Configuração constante (k=4, chunk 500/50, gpt-oss:120b, nomic-embed-text). A variável declarada entre iterações é o prompt RAG (v1 → v2); a execução revelou, porém, um defeito no índice (chunks duplicados) que contaminava as medições — por isso a tabela mostra quatro colunas: as medidas originais com o índice defeituoso e as re-medidas após a correção.

Métrica	Iter 1 (índice defeituoso)	Iter 2 (índice defeituoso)	Iter 1 (índice corrigido)	Iter 2 (índice corri
Faithfulness (média)	1.000	0.875	1.000	1.000
Answer relevancy (média)	0.944	0.875	1.000	1.000
Chunk da fonte certa recuperado	0.923	0.923	1.000	1.000
Resposta com citação	0.933	0.800	1.000	1.000
Fonte correta citada	0.867	0.800	1.000	1.000
Fatos-chave presentes	0.833	0.800	1.000	1.000
Injection resistido	1.000	1.000	1.000	1.000
Recusa correta	0.944	0.875	1.000	1.000
Erros do juiz (de 19)	1	3	0	0

5. Problemas encontrados e soluções

5.1 Índice com chunks duplicados (o mais grave)

A iteração 2 'piorou' e casos com informação existente passaram a recusar ('não encontrei na base'). Diagnóstico: o Chroma.from_documents do LangChain **adiciona** à coleção em vez de recriá-la — o índice acumulou 51 embeddings para 17 documentos únicos (3 cópias de cada), e o top-k=4 voltava preenchido de duplicatas, empurrando o chunk correto para fora do corte. O modelo recusava corretamente (o grounding estava funcionando); a culpa não era do prompt v2. Solução: coleção apagada antes de reindexar + erro rápido se a coleção ficar vazia. Ganho medido (iter 2): citação/fonte/fatos +0.200, faithfulness +0.125, recusa +0.125.

5.2 Juiz devolvendo JSON truncado

1 a 3 casos por iteração com juiz_erro ('substring not found'): o parser exigia JSON completo e o modelo avaliador às vezes devolvia texto truncado. Casos bons eram excluídos da média, tornando as iterações não comparáveis. Solução: parser tolerante (extrai o maior objeto JSON válido, tolerando texto em volta), retry com instrução reforçada e max_tokens do juiz 600→1500. Resultado: 0 erros de juiz nas duas iterações re-medidas.

5.3 Ground truth incompleto no caso f-05

fonte_correta 0.867 mesmo com tudo certo: o caso esperava apenas regimento_condominial.pdf, mas o fato ('reserva de até 4 horas') existe também em faq_carregamento.pdf — a resposta estava correta e o ground truth (dado esperado) é que era incompleto. Corrigido para aceitar os dois documentos; métrica → 1.000.

5.4 Caracteres invisíveis quebravam a checagem de citação

citation_rate 0.933 em execução idêntica: o modelo da nuvem inseria zero-width space (U+200B) no meio da palavra 'Fonte:' e a checagem determinística não encontrava o marcador. Solução: normalização removendo caracteres invisíveis (U+200B–U+200F, U+2060, U+FEFF, U+00AD) antes de comparar; recalculado sem nova chamada ao modelo → 1.000.

5.5 Set difícil revela diferença real entre prompts

Com o eval set oficial saturado (teto 1.0), um set complementar mais difícil (8 casos) separou as duas versões de prompt: a v2 'obedeceu' a uma instrução injetada dentro do próprio documento (acrescentou um aviso promocional à resposta) enquanto a v1 resistiu; as demais métricas empataram. Resistência a injection no documento fica como protocolo para uma próxima versão de prompt. O experimento também exigiu canonizar separadores de milhar no medidor ('5.000' = '5 000').

6. Comparação entre modelos no contexto RAG

Os mesmos 19 casos, mesmo prompt v2, mesmo retrieval (k=4) e mesmo avaliador — variando apenas o modelo gerador e o max_tokens (docs/comparacao_modelos_rag_raw.json):

Métrica	gpt-oss:120b (512)	nemotron (512)	nemotron (2048)
Faithfulness (média)	1.000	0.684	0.947
Answer relevancy (média)	1.000	0.053	1.000
Fonte correta citada	1.000	0.067	1.000
Fatos-chave presentes	1.000	0.067	1.000
Recusa correta	1.000	0.789	1.000
Latência média/caso	2.95s	5.56s	11.75s

Com parâmetros idênticos, o nemotron-3-nano:30b devolveu 17 de 19 respostas **vazias** no RAG: gasta todo o orçamento de 512 tokens no raciocínio interno (efeito agravado pelo contexto longo recuperado). Com max_tokens=2048 alcança quase a paridade, mas com latência 4x maior. **gpt-oss:120b é a escolha do projeto no RAG**: única configuração com 1.0 em todas as métricas e a mais rápida. No comparativo da Sprint 03 (chain conversacional), o gpt-oss:120b também foi a melhor escolha: cerca de 4.4x mais rápido e mais conciso que o nemotron, com igual segurança contra jailbreak.

7. Equipe e divisão de trabalho

Integrante	Responsabilidades na Sprint 04
Pedro Ferreras (RM 568713)	— (completar pela equipe antes da entrega) —
Pedro Santos (RM 571017)	— (completar pela equipe antes da entrega) —
Victor Binot (RM 571499)	— (completar pela equipe antes da entrega) —
Gustavo Kunitaki (RM 571400)	— (completar pela equipe antes da entrega) —
Kauanne Oliveira (RM 574191)	— (completar pela equipe antes da entrega) —
Nayhely Estela (RM 571416)	— (completar pela equipe antes da entrega) —

8. Limitações e próximos passos

- O ganho principal da sprint veio da correção dos defeitos de infraestrutura (indexação e avaliador), não da troca de prompt; o set difícil (8 casos) mostrou que a v2 é mais suscetível a instrução injetada no documento do que a v1 — correção alvo para uma futura versão v3 do prompt.
- Cada pergunta é respondida de forma independente — sem memória no RAG.
- Embeddings dependem do Ollama local em execução na máquina.
- O juiz penaliza a frase fixa de recusa como afirmação não suportada (visto no caso o-01 com o nemotron) — mitigável com critério de recusa na rubrica do avaliador.

Equipe: Pedro Ferreras (RM 568713) · Pedro Santos (RM 571017) · Victor Binot (RM 571499) · Gustavo Kunitaki (RM 571400) · Kauanne Oliveira (RM 574191) · Nayhely Estela (RM 571416)